# Análise Sintática

Atividade - Tratamento de Erros em Analisadores Descendentes Recursivos

Implementar tratamento de erros sintáticos segundo o modo pânico para a linguagem da GLC dada:

**<read>** → read identificador

**<write>** → write identificador

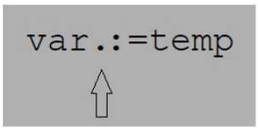
**<atribuição>** → identificador := **<exp aritmética>**

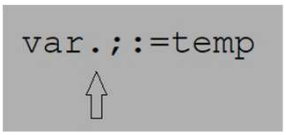
**<exp aritmética>** → **<operando>** {**<op aritmético>** **<operando>**}

**<operando>** → identificador | num

**<op aritmético>** → + | *

# Conjunto Sequência




- Caso não haja match:
- Uma solução plausível é solicitar o próximo token ao analisador léxico até que haja:
  - Match com a marca esperada (ie: ":=")
  - O fim da linha.
- Esta lógica é baseada na ideia de que houve a inserção de um ou mais tokens inesperados, mas que na sequência os tokens estão corretos (conforme a regra da GLC).

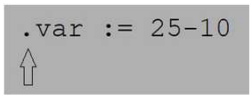
# Conjunto Sequência

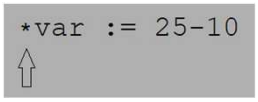
- Composto por todos os tokens (terminais) que sucedem um determinado símbolo da gramática (seja ele variável ou terminal).
- No tratamento de erros, o interesse é no conjunto sequência apenas de símbolos terminais, pois são eles que estão no código fonte sendo analisado.

**<read>** → read identificador
**<write>** → write identificador
**<atribuição>** → identificador := **<exp aritmética>**
**<exp aritmética>** → **<operando>** {**<op aritmético>** **<operando>**}
**<operando>** → identificador | num
**<op aritmético>** → + | *

SEQUÊNCIA (read) = {identificador}
SEQUÊNCIA (write) = {identificador}
SEQUÊNCIA (:=) = {identificador, num}
SEQUÊNCIA (identificador) = {+,*, :=, $}
SEQUÊNCIA (num) = {+, *, $}
SEQUÊNCIA (+) = {identificador, num}
SEQUÊNCIA (*) = {identificador, num}

## Conjunto Primeiros




- Caso não haja match:
- Uma abordagem plausível é considerar tokens que iniciam instruções como sendo de sincronização.

| Regra que gera instrução da linguagem | Regra de GLC que não gera instrução da linguagem |
|---|---|
| **<atribuição>** → identificador := **<exp aritmética>** | **<exp aritmética>** → **<operando>** {**<op aritmético>** **<operando>**} |

- O conjunto Primeiros dos símbolos variáveis são relevantes porque alguns destes símbolos variáveis especificam instruções da linguagem.
- Para estes símbolos variáveis, interessa saber quais são todos os terminais que podem iniciar estas instruções, ou seja, o conjunto primeiros destes variáveis.

## Conjunto Primeiros

- O conjunto primeiros (de um símbolo variável) contém todos os terminais que iniciam formas sentenciais derivadas por aquele símbolo variável.
- No tratamento de erros o interesse é no conjunto primeiro de símbolos variáveis que geram as instruções da linguagem.

**<read>** → read identificador
**<write>** → write identificador
**<atribuição>** → identificador := **<exp aritmética>**
**<exp aritmética>** → **<operando>** {**<op aritmético>** **<operando>**}
**<operando>** → identificador | num
**<op aritmético>** → + | *

PRIMEIROS(<read>) = {read}
PRIMEIROS(<write>) = {write}
PRIMEIROS(<atribuição>) = {identificador}

# Descrição:

- Implementação do reconhecimento de erros sintáticos (modo pânico).
- A saída é "declaração sintaticamente correta" caso a declaração não possua erros sintáticos ou , caso haja erros, deve imprimir mensagens de erro apropriadas, bem como o token que causou o erro.
- Exemplo de declaração:
  - read 20
- Exemplos de mensagens de erro:
  - Erro "20". Esperado ID
- Exemplo de declaração:
  - 1 read var
- Exemplos de mensagens de erro:
  - Erro "1". Esperado ID ou palavra reservada (“read” ou “write”).